Future Generation Computer Systems 150 (2024) 354–363

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

DAG-based workflows scheduling using Actor–Critic Deep Reinforcement Learning

Guilherme Piêgas Koslovski *, Kleiton Pereira, Paulo Roberto Albuquerque

Graduate Program in Applied Computing, Santa Catarina State University, Brazil

ARTICLE INFO

Article history:
Received 15 June 2023
Received in revised form 24 August 2023
Accepted 11 September 2023
Available online 15 September 2023

MSC:
00-01
99-00

Keywords:
Scheduling
Actor–critic
Deep reinforcement learning
DAG
Tasks
Jobs
Workflow

ABSTRACT

High-Performance Computing (HPC) is essential to support the advance in multiple research and industrial fields. Despite the recent growth in processing and networking power, the HPC Data Centers (DCs) are finite, and should be carefully managed to host multiple jobs. The scheduling of tasks (composing a job) is a crucial and complex task, once the reflexes of the scheduler's decisions are perceptible both for users (e.g., slowdown) and for infrastructure administrators (e.g., use of resources and queue length). In fact, the process of scheduling workflows atop a DC can be modeled as a graph mapping problem. While an undirected graph is used to represent the DC, a Directed Acyclic Graph (DAG) is used to express the tasks dependencies. Each vertex and edge from both graphs can have weights associated with them, denoting the residual capacities for DC resources, as well as computing and networking demands for workflows. Motivated by the combinatorial explosion of the aforementioned scheduling problem, the integration of Machine Learning (ML) for generating or improving scheduling policies is a reality, however the proposals in the specialized literature opt, mostly, for using simplified models to reduce the search space or are trained to specific scenarios, which leads to policies that eventually fall short of real DCs expectations. Given this challenge, this work applies Actor–Critic (AC) Reinforcement Learning (RL) to schedule DAG-based workflows. Instead of proposing a new policy, the AC RL is used to select the appropriated scheduling policy from a pool of consolidated algorithms, guided by the DAGs workload and DC usage. The AC RL-based scheduler analyzes the DAGs queue and the DC status to define which algorithms are better suited to improve the overall performance indicators in each scenario instance. The simulation protocol comprises multiple analysis with distinct workload configurations, number of jobs, queue ordering polices and strategies to select the target DC servers. The results demonstrated that the AC RL selects the scheduling policy which fits the current workload and DC status.

1. Introduction

Context and motivation. Multiple complex applications depend on High-Performance Computing (HPC) infrastructures to achieve the expected results. Examples of applications come from different areas of knowledge, including simulations on the use of wind farms, computational fluid dynamics, fusion power, earthquakes, quantum chemistry, machine learning, among others [1, 2]. In common, the request to execute some of the aforementioned applications is represented as a job composed of one or more tasks. Following a graph notation, a job is represented as an heterogeneous Directed Acyclic Graph (DAG), in which vertices denote the tasks and edges denote data transfers between tasks, characterizing the dependency constraints between tasks. Furthermore, weights are added to vertices and edges to indicate, respectively, the processing and communication requirements between a pair of tasks. In turn, the HPC infrastructure (e.g., supercomputer, DC) is abstracted as another graph, where the vertices are the computing resources (e.g., servers, processors) and edges represent the communication links and paths. Weights in vertices and edges represent the available residual processing and communication capacity, respectively.

Despite the technological evolution of HPC DCs that has occurred in recent decades, some management challenges remain as important research topics to enhance the use of such infrastructures, as many jobs can simultaneously run on a DC. Specifically, workflows scheduling in HPC belongs to the *NP-hard* problem set, as it can be reduced to other proven scenarios from this set [3–7]. In short, the scheduler must order tasks from multiple DAGs considering the residual DC processing and communication capacities.

* Corresponding author.
E-mail addresses: guilherme.koslovski@udesc.br (G.P. Koslovski), kleiton.pereira@edu.udesc.br (K. Pereira), paulo.albuquerque@edu.udesc.br (P.R. Albuquerque).

https://doi.org/10.1016/j.future.2023.09.018
0167-739X/

It is notable the challenges for composing scheduling algorithms, moreover for analyzing requests representing large-scale applications (in form of DAGs) that must be allocated atop DCs composed by thousands of servers [8,9]. In fact, traditional First Come First Served (FCFS), Shortest Area First (SAF), Shortest Processing Time first (SPT), Shortest Number of Processors First (SQF), or any other complex jobs queue policies [10–12] must be jointly executed with DC servers' selection policies (e.g., tasks consolidation and spreading) [13] to satisfy both DC administrators and users objectives. The specialized literature (discussed in Section 5) comprises several heuristics with manually-tuned parameters that, although efficient in their application domains, partially solve the scheduling of DAGs, usually ignoring the ordering of DC servers. It is not trivial to define the perfect match of queueing policy, servers ordering, and workflows characteristics, and it is common the use of techniques to reduce the search space, such as resource grouping, network abstraction, among others [13,14].

Research hypothesis and the idea. In recent years, the use of ML has become increasingly pronounced in the management of distributed infrastructures and task scheduling [15–20] due to its dynamic ability to recognize patterns and aid in the decision-making process [21]. In this context, ML is used mainly in two scheduling-related tasks: the creation of new queueing policies and to improve the state-of-the-art policies tailoring the performance to specific workloads. However, it is important to note that although the state-of-the-art demonstrated the applicability of ML for scheduling tasks in different contexts [20,22–25], two crucial aspects still need to be explored for full use in HPC: *(i)* currently, the objectives of DC administrators and users are individually analyzed, whereas in real scenarios, it is necessary to compose policies that act simultaneously with both perspectives; and *(ii)* the pursuit of novel scheduling policies grounded in ML necessitates a perpetual process of readapting and reevaluating the model to effectively align with the diverse workloads and dynamic status of DCs. We argue that instead of proposing new workload-tailored policies, we should find the appropriated mapping between existing state-of-the-art policies and scheduling objectives. Specifically, in DCs hosting heterogeneous workloads, each job represents an application with distinct requirements in terms of communication and processing parameters, and all these characteristics create an environment that demands careful choices of alternatives to select the appropriate scheduling policy and where there is no *silver bullet* scheduler, which is the best for all cases.

Our contribution. Given this fact, this work jointly analyzes workloads and DC status to select the appropriate scheduling policy from a pool of consolidated proposals from specialized literature. We use AC RL, which requires little computational effort to take an action over continuous values [26,27], to select the scheduler for each discrete time interval. Each action taken by the AC RL model selects two scheduling policies: one for ordering the workflows queue and another for selecting the DC servers. Essentially, RL fits the DAG-based workflows scheduling in HPC DCs as this approach can learn from continuous experience, without the need of a predefined and labeled database. Specifically, the AC RL method is able to infer better decisions directly from the current DC status and queued tasks. The resulting scheduler was assessed in our experimental protocol to schedule scenarios varying the number of jobs, the workflow format, and distinct workload configurations (termed graph connectivity). The results indicate that even when varying the load and the scenario, the model finds the appropriate combination of schedulers for most cases.

Organization. This paper is organized as follows. The problem is formulated in Section 2, while Section 3 presents the scheduler's model, architecture, and prototype details. An evaluation is presented in Section 4, and related work is discussed in Section 5. Finally, Section 6 lists our final remarks and future work.

2. Problem definition

Our problem formulation uses weighted graphs, DAG models and graph embedding algorithms. Initially, jobs and HPC DC are modeled as graphs in Section 2.1, while the main metrics to represent users and HPC infrastructure perspectives are presented in Section 2.2. Section 2.3 focus on tasks dependencies and temporal aspects. Finally, we present the jobs queueing policies and the DC objectives in Section 2.4.

2.1. Jobs, tasks, and HPC DCs

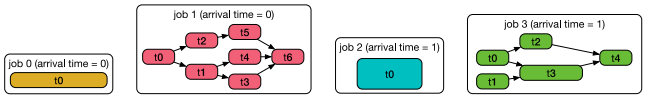
There are numerous variations of the HPC scheduling problem, however, for the present work we will stick to the specific variation that defines each job as a set of operations, or tasks, which must be processed in a predetermined order (composing a workflow). This variation of the problem also introduces the concept of precedence constraint, or dependency. In fact, when respecting the temporal limitations generated by dependencies, two or more tasks may be performed in parallel, decreasing the overall makespan (the length of time that elapses from the start of the first task to the jobs' end) of that specific job. For illustrating the scenario independently of HPC DC size, Fig. 1 presents four jobs with distinct compositions. Each rectangle represents a task: processing is denoted by the rectangle height and execution time by the base's length. Jobs 0 and 2 from Figs. 1(a) and 1(c), respectively, are composed by a single task, however, the tasks have different processing and execution time requirements. In turn, jobs 1 and 3 (Figs. 1(b) and 1(d), respectively) have detailed workflows, as denoted by the arrows. Finally, the jobs arrival time is defined as the first submission time from composing tasks.

Formally, a set of jobs (termed $\mathcal{S}$) must be scheduled for executing atop a HPC DC following an arrival time distribution. Each job j is represented as a DAG $G_j(V_j^G, E_j^G)$ in which V_j^G denotes the vertices of the graph, in other words, the N tasks that demand computational resources. In turn, an edge $e_{mn} = (m, n) \in E_j^G$ denotes a data transfer that starts with the task m and ends in n and it may only happen after m's execution has been completed, characterizing the temporal sequence of execution. Following the graph's notation, the HPC infrastructure is abstracted as a graph $H(V^H, E^H)$, where V^H represents the computing resources (e.g., servers, processors) and E^H the communication links and paths. Furthermore, weights are added to vertices and edges to indicate, respectively, the processing and communication requirements for DAGs, and the capacities for the DC graph ($c_s; s \in V^H$ denotes the original capacity and $c'_s; s \in V^H$ indicates the residual one).

When submitting a job j, each composing task $m \in V_j^G$ must detail three parameters traditionally used by Resources and Jobs Management Systems (RJMSs) from HPC infrastructures: the estimated processing time ($\tilde{p}_m$, also termed as walltime), the processing resource requirement (q_m), and the time in which the task was submitted to the RJMS (r_m). Later, the RJMS appends the start time (s_m) defined by the scheduler, and the actual processing time (p_m, only know after the task execution) for each task. Finally, to schedule a job atop DC resources, the procedure is reduced to a graph map problem with dependency and time constraints.

2.2. Metrics and perspectives

We focus on a scheduling scenario classified as open, with online submissions, and among the multiple metrics that are available to represent users and DC administrator perspectives [28], we focus on bounded slowdown (bsld) for representing users' jobs, while investigate the servers' footprint and queue length for denoting DC's perspective.



(a) 1 task. (b) 7 tasks. (c) 1 task. (d) 5 tasks.

Fig. 1. Examples of tasks and jobs with distinct arrival times and demands (in terms of processing and expected walltime). Each rectangle represents a task: processing is denoted by the rectangle height and execution time by the base's length. The arrows represent the execution's workflow.

The waiting time w_m of each task $m \in G_j^V$ is accounted as $w_m = s_m - r_m$ and denotes how long a task waited before starting its execution. Following this line, the bounded slowdown for a given task m is given by $bsld_m = max\left\{\frac{w_m+p_m}{max(p_m,\tau)}, 1\right\}$, where τ is a constant that is typically set in the order of 10 s, that prevents small tasks from having excessively large slowdowns [11,28].

In turn, the queue length is constructed as a cumulative function, summing up the number of tasks that are waiting to be scheduled at a given time instant. It is worthwhile to mention that a job is not considered an atomic entity, and the tasks composing multiple jobs simultaneously share the infrastructure resources. In this sense, tasks can be anticipated or delayed, and eventually can lead a job to a starvation scenario (when this condition is not treated by the RJMS). Formally, at a given time instant t, all tasks composed with $r_m \leq t$ are eligible to be scheduled and populate the queue until be processed by the RJMS. Tasks postponed due to lack of DC resources at time instant t remain queued to be processed in the next time events (at this point the starvation phenomena may be observed).

2.3. Dependencies and temporal aspects

Scheduling DAGs atop HPC DCs resources with communication and temporal requirements introduces a new complexity dimension. Instead of verifying the DC residual capacity at a given time instant, the scheduler must analyze a time window of possibilities, considering both processing and networking capacities. Specifically, the time window starts at the job's arrival time ($min\{r_m\}|m \in G_j^V$), which is eventually increased when the tasks are queued due to lack of resources at that point. However, the maximum size suitable for the time window is not known in advance. It should be large enough to embrace the sum of the tasks' walltime for tasks composing the DAG's critical path (the DAG's depth). Taking jobs 1, 2, and 3 from Fig. 1, the minimum window size for analyzing both jobs is composed of 4 discrete events, considering the discrete event equivalent to the small-sized task. In this sense, the scheduler must find appropriated slots for scheduling all tasks composing the jobs respecting the predefined time window, otherwise the internal slowdown is increased, as well as the job's makespan.

To standardize the parameter among all jobs, an alternative is to use a previously defined configuration. In this sense, usually the RJMSs have a predefined maximum walltime for reservation (24 h, for example), which dictates how long a job can run atop the HPC DC. This information can be used as a maximum value for all DAGs.

2.4. Tasks queueing policies and DC's objectives

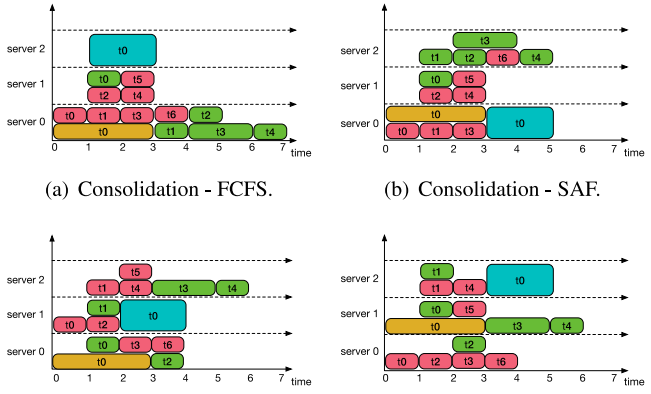
HPC queueing policies (and eventually parameters' tuning) have been largely investigated by the specialized literature (as discussed in Section 5). First Come First Served (FCFS), Shortest Processing Time first (SPT), Shortest Number of Processors First (SQF), and Shortest Area First (SAF) are examples of popular

Table 1
Task-based policies and their parameter formulas.

Policy	Formula
FCFS	r_m
SPT	p_m
SQF	q_m
SAF	$p_m \cdot q_m$
F1	$\log_{10}(p_m) \cdot q_m + 8.70 \cdot 10^2 \cdot \log_{10}(r_m)$
F2	$\sqrt{p_m} \cdot q_m + 2.56 \cdot 10^4 \cdot \log_{10}(r_m)$
F3	$p_m \cdot q_m + 6.86 \cdot 10^6 \cdot \log_{10}(r_m)$
F4	$p_m \cdot \sqrt{q_m} + 5.30 \cdot 10^5 \cdot \log_{10}(r_m)$

Table 2
DC-based policies and their parameter formulas.

Policy	Formula
Consolidation	c_s'
Spreading	$c_s - c_s'$



(a) Consolidation - FCFS. (b) Consolidation - SAF.

(c) Spreading - FCFS. (d) Spreading - SAF.

Fig. 2. Examples of scheduling policies for all jobs from Fig. 1. The consolidation policy uses a best-fit approach to decrease the DC's fragmentation (activation of servers and network equipment), while the spreading policy prioritizes the increase of reliability by placing tasks atop distinct DC servers, rack, and other group definitions.

policies, having in common the users (jobs) orientation, aiming at improving users' perspectives (as discussed in Section 2.2). In parallel, efforts have been made to improve the administrative DC's perspective by consolidating the resources usage, decreasing the energy consumption, maximizing the overall infrastructure throughput, among others objectives. Recently, proposals using ML have been successfully applied to infer new scheduling policies [29]. Essentially, each new policy aims at maximizing a new objective's function guided by a set of tasks parameters. While the task-based policies are driven by tasks properties (as shown in Table 1), the DC-oriented policies use the infrastructure's information (as seen in Table 2). However, developing a new DAG scheduling policy aware of both objectives is a challenging task, which eventually include dichotomies in decision-making.

We argue that both perspectives must be jointly analyzed, as exemplified by Fig. 2. The DC graph H is composed of three homogeneous servers that must process the jobs from Fig. 1. To exemplify the discussion, the scenario comprehends four combinations of queueing policies (FCFS and SAF) and DC objectives (consolidation and resource spreading). At each time instance, the consolidate approach aims at scheduling the maximum number of tasks atop already active servers, decreasing the fragmentation metric while increasing the overall DC footprint. In turn, the spreading of jobs atop DC resources is a technique used to increase reliability and survivability metrics (not investigated in

this work). The selection of DC resources spreading or consolidation policy has an immediate impact on data traffic [13]. The consolidation of tasks atop a single subset of DC resources can increase the overall network performance perceived by the application, however this approach can also lead to local congestion issues [30]. In turn, the spreading of communication tasks atop DC resources tends to press the core links, which connect the racks and servers, leading to a degradation of overall network performance [8].

Numerically, the examples of Fig. 2 demonstrate that the combination of FCFS with DC consolidation decreases the waiting time of job 2 when compared to others policies combinations, while the joint execution of DC consolidation and SAF decreases the total makespan. The impact of policies choice on DC fragmentation's metric is perceived after the time interval 3. Specifically, the combination of SAF with DC spreading required the action of three servers, while FCFS combined with DC spreading or consolidation policies used just a single server. The slowdown metric is also impacted by the queueing policies combination, as observed by the scheduling of jobs 2 and 3. Finally, spreading tasks atop servers tends to increase the overall network load, leading to congested scenarios [13].

We claim that the scheduler must be sensitive for both users and DC objectives, adapting the decision-making process according to jobs workloads and DC usage. Instead of using only a subset of policies, the scheduler must, in long-term, define the appropriated policies combination to schedule jobs. That is exactly the proposal of the present work, the use of Actor–Critic (AC) Reinforcement Learning (RL) for composing a scheduling policy aware of the evolving nature of DC and jobs requirements. Instead of proposing new queueing policies, the AC RL model learns which policies must be used to schedule any specific queue, aware of jobs workloads and DC residual capacities.

3. Actor–Critic RL-based scheduler

The traditional Deep Reinforcement Learning (DRL) method uses an iterative algorithm based on a subset of ML techniques called neural networks, which keeps on modifying its internal model to improve the decision making. The training data of DRL generates a set of state–action pairs and rewards, and the agent receives the feedback of its actions applied to the environment, learning how to perform better by continuously updating its model's internal weights. The goal of the DRL algorithm is to find the optimal policies combination in which it specifies the correct action to take in each possible state, maximizing the reward. It is a fact that DRL does not have an immediate optimal performance but instead keeps enhancing it in the long run, which makes it even better suited for decision making, planning, and scheduling scenarios which evolve over time.

Specifically, the AC method employs a separate structure to represent the policy independently of the value function. The policy structure is known as the actor, as it is the one to take actions while the value function is the critic, as it criticizes the actions taken by the actor, which comes out as the reward of each action. In this method, the critic must learn how to criticize to later critique the actions the policy follows. In this context, we describe the AC scheduler's components (Sections 3.1, 3.2, and 3.3), and later present details on the training and learning (Section 3.4) process as well as on prototype implementation (Section 3.5).

3.1. Environment

The environment partially represents the internal structure of a traditional RJMS, combining information from the DC graph and jobs (running and waiting). Processing and networking resources are represented as profiles, controlling the allocated and residual capacity at each time event. In addition, a scheduling map is populated with scheduled workflows, filling up the jobs and tasks metrics (from Section 2.1). Essentially, the environment acts as a ledger by tracking the schedulers' decisions, DC usage, and jobs submissions.

3.2. Data input and output

Our approach opts for calling the decision-making for each discrete time interval (as exemplified by Fig. 2). The AC scheduler receives as input details on waiting tasks and the residual capacity of DC components. Specifically, data input is defined as a vector with values $\{x \in \mathbb{R} \mid 0 \geq x \leq 1\}$ with a predefined upper-bound size based on queue and DC elements. For each task $m \in V_j^G$, we selected four properties: the estimated processing time ($\tilde{p}_m$), the processing resource requirement (q_m), and concatenated details on graph workflow (vertex in-degree and out-degree, representing dependencies and dependents). Both degrees are normalized by the number of tasks composing a workflow, while the other properties are normalized using the maximum values. Later, the DC status obtained from the environment is appended. Specifically, each server's capacity profile (representing the available residual capacity over time) is represented with minimum, average, and maximum values, regarding the time-window under analysis (the window size must be large enough to accommodate the minimal makespan of a DAG).

It is worthwhile to mention that for training and using the AC-based scheduler, we defined a fixed number of tasks to compose the input vector, termed slice. Theoretically the queue size is unlimited, however the computing resource executing the scheduler (Graphics Processing Unit (GPU) or server, for instance) has a memory limit that acts as an inflexible boundary. In other words, the maximum slice's size of given queue is directly related with the available memory that can be efficiently manipulated by the environment hosting the AC RL scheduler. When the scheduler is called, the task queue is then split into slices, that are individually analyzed by the scheduler. Eventually, the input vector is padded with zero values to keep size consistency. Finally, all values are normalized by its correspondent maximum values to fit the vector's domain.

The AC RL scheduler offers two outputs, denoting the actor's action and the critic's perspective. The actor's goal is to discover and indicate the appropriated combination of policies for sorting the task queue and the DC resources. The output's vector represents all permutations among both sets, using FCFS, SPT, SQF, SAF, F1, F2, F3, and F4 as choices for tasks ordering, and consolidation and spread for DC resources. In this sense, the actor must indicate the choice among 16 possible options, represented by dense layers activated with a softmax function. Finally, the critic should evaluate the actor's choice, and this action is represented by a dense layer with a single output value, representing the critic's opinion on the actor's performance. Details on layers compositions are given in Section 3.5.

3.3. Reward function

The AC RL scheduler aims at selecting the appropriated combination of queueing policies to maximize the cumulative reward function. Our reward function is based on reducing the overall bounded slowdown (bsld) metric, which is usually applied as primary objective for state-of-the-art policies (related work is discussed in Section 5), combined with an incentive for increasing the number of scheduled tasks (and consequently the DC usage). For each discrete time event (termed t), the reward function ($\mathcal{R}(t)$) is accounted as given by Eq. (1), where $\mathcal{M}(m, t) \in \{0, 1\}$

is a binary mapping function, indicating if task m is scheduled at time t. Essentially, for maximizing the reward we have to decrease the bounded slowdown and increase the number of scheduled tasks, considering the maximum allowed time $window$ (the same time window used for composing the input data in Section 3.2).

$$\mathcal{R}(t) = \int_{t}^{t+window} \sum_{j \in S} \sum_{m \in V_j^G} \frac{\mathcal{M}(m,t)}{blsd_m} dt \quad (1)$$

The rationale for composing the reward function is to give incentives for increasing the acceptance ratio while finding efficient scheduling solutions. However, our simulation framework is extensible, and other reward functions can be adapted.

3.4. Training and learning

The AC RL method learns from multiple interactions with the environment, collecting the perspective of actor and critic neural networks [31]. In this sense, AC RL can learn from an unlabeled dataset, differentiating the action's efficiency based on a predefined reward function. Specifically, our proposal uses algorithms from the consolidated literature, evaluating their efficiency through a single reward function. For training the AC RL model, a dataset of DAGs is analyzed multiple times (the training episodes – algorithms are discussed in Section 3.5), discovering the efficient algorithm to schedule a given combination of HPC DC status and tasks queue.

Instead of executing all combinations of scheduling algorithms for each pair of DC status and tasks queue, the actor selects just one option among all available, executes it, and receives a reward for its action. Once the AC RL executes multiple interactions on a single dataset, in long term, the reward function will guide the selection to the appropriated combination of queue policy and DC resources ordering. In summary, the AC RL method can adapt and generalize the decision-making to the DC load and tasks queue (the AC RL efficiency is discussed in Section 4).

About training costs of an AC RL model, it is necessary to define that they are directly related to the volume of data, number of iterations and algorithm options available. The training process is considered complete when the actions taken by the actor fail to result in a significant increase in the obtained reward. However, it is important to emphasize that the learning process can be resumed at any time, whenever a significant variation in the environment is observed (e.g., a distinct workload, an update on HPC DC topology). Finally, in production, an already trained model quickly decides (similar to querying an indexed table) which combination of tasks queue and DC resources ordering should be used, at each discrete event, improving the overall performance indicators. At this point, the system bottleneck remains the original algorithmic complexity of the selected policies.

3.5. Prototype implementation

We implemented the scheduler's prototype in Python version 3.10.7 using TensorFlow 2.10.0 as the ML backend framework. The scheduler model was compiled with the Adam optimizer [32] (using a loss rate of 0.01), and Huber loss was used as loss function for the critic. In addition to the input and output layers discussed in Section 3.2, two densely-connected layers with 64 and 32 units were used for composing the AC RL prototype.

The pseudocode for training the AC RL scheduler is presented in Algorithm 1. The algorithm receives as input parameters a set of DAGs representing the jobs (termed $\mathcal{S}$) that must be allocated, a graph denoting the HPC DC (termed H), the number of discrete time events to train the model (termed $\mathcal{T}$), the number of training episodes (termed $\mathcal{E}$), and two parameters used to control the task queue (the lookahead $window$ and the maximum number of tasks per slice - *tasks_limit*). The lookahead window defines how far in time the scheduler should analyze the capacity profiles from DC resources for trying to schedule the tasks, while the tasks limit is essential to generalize the model application in multiple scenarios.

The environment and neural network model are created at lines 1 and 2 following the discussion from Sections 3.1 and 3.2, respectively. The training process is executed in episodes (line 5). For each episode, the jobs and DC status are restored to the initial condition (lines 6 and 7), followed by all discrete events being sequentially executed (line 8). All jobs submitted before the current event populate the tasks queue (line 9), however the tasks are processed in slices (line 10). At this point, a queue slice is translated into a state representation (line 11) following the discussion from Section 3.2, and submitted as input to AC RL model (line 12), which defines the scheduler selection and the critic's perspective. This information is given as input for the environment (line 13), which executes the selected scheduling policies and updates the internal state (DC, jobs, and tasks status) as well as quantifies the action's reward, following the reward function presented in Section 3.3. While training, a set of historical data is accounted (lines 3, 4, 14, and 15) preparing the back propagation process for training both the actor and critic. Rewards are summed up considering the discount value for past rewards, termed γ (with value 0.99) updating the historical data (lines 20 to 22). Finally, the losses are calculated and a back propagation approach is used to update the gradients values (lines 24 to 30, specifically, the function $agg()$ performs an one-to-one map among all sets). After training, the resulting model (and its internal weights) are saved to be later used as an independent scheduler.

Motivated by the discussion from Section 2.4, our prototype considers the combination of two sets of policies, one for ordering tasks and other for DC resources. In this sense, the scheduler's selection (line 12) actually defines two policies, which are used for decision-making. While the policies are representative examples applied by the specialized literature, they are fundamental to demonstrate that AC RL can learn which algorithms should be combined for scheduling a set of tasks. In fact, the prototype can be extended by adding new policies. The pseudocode from Algorithm 2 summarizes the scheduling process (invoked when the environment is performing a step - line 13).

The Algorithm 2 receives as inputs the discrete time event being processed (termed $t \in \mathcal{T}$), the *scheduler* configuration selected by the AC RL model, a slice of jobs (termed $\mathcal{J} \in \mathcal{S}$), the current DC status, and the lookahead $window$. The *scheduler* configuration is decomposed (line 1) into *queue_policy* and *dc_policy* representing the policies that must be used for ordering tasks and DC resources, respectively. All unscheduled tasks from $\mathcal{J}$ with submission time lower than the current time event t are selected to be analyzed (in other words, $\forall j \in \mathcal{J}; \forall m \in V_j^G; r_m \leq t \wedge \neg\mathcal{M}(m,t)$). After composing the tasks queue, the selected policy is applied (line 3), and all tasks are individually analyzed (lines 4 to 11). For scheduling a DAG, the algorithm uses the lookahead window (lines 5 to 10), and for each event (termed ev), the pre-selected DC resource ordering policy is used (line 6). The algorithm sequentially analyzes all DC candidates to host the task m (line 7). We opted for selecting the first-fit map, interrupting the search, and updating the DC status and the controlling map $\mathcal{M}$. It is worthwhile to mention that the Algorithm 2 is a proof-of-concept prototype, and could be replaced by any specialized (and eventually complex) DAG scheduling algorithm from the specialized literature. Finally, once the AC RL model is trained and saved, it can be incorporated in any existing RJMS, simulator, and other scheduling tools.

Algorithm 1: AC-based scheduling.

Input: A set of jobs $\mathcal{S} = \{G_1, G_1, \ldots, G_n\}$
Input: A HPC DC $H(V^H, E^H)$
Input: Scheduling time events ($\mathcal{T}$)
Input: Number of training episodes ($\mathcal{E}$)
Input: Lookahead window size ($window$)
Input: Number of tasks per processing ($tasks\_limit$)
$env \leftarrow environment(H, \mathcal{S}, window)$;
$ac\_rl \leftarrow create\_model(H, \mathcal{S}, \mathcal{T}, window, tasks\_limit)$;
$history \leftarrow \emptyset$;
$rewards \leftarrow \emptyset$;
for $episode \leftarrow 1$ ***to*** $\mathcal{E}$ **do**
  $env.reset\_jobs()$;
  $env.reset\_dc()$;
  for $event \leftarrow 1$ ***to*** $\mathcal{T}$ **do**
    $jobs \leftarrow jobs\_by\_event(\mathcal{S}, event)$;
    for $slice \in get\_slices(jobs, window, tasks\_limit)$ **do**
      $state \leftarrow env.get\_state(slice)$;
      $scheduler, critic \leftarrow ac\_rl.schedule(state)$;
      $reward \leftarrow env.step(scheduler, slice, event)$;
      $history.insert([scheduler, critic])$;
      $rewards.insert(reward)$;
    end
  end
  $returns \leftarrow \emptyset$;
  $rewards\_sum \leftarrow 0$;
  for $r \in reward$ **do**
    $rewards\_sum \leftarrow r + \gamma \cdot rewards\_sum$;
    $returns.insert(rewards\_sum)$;
  end
  $returns \leftarrow \frac{returns - mean(returns)}{std(returns)}$;
  $losses \leftarrow \emptyset$;
  for $scheduler, critic, reward \in agg(history, returns)$ **do**
    $diff = reward - critic$;
    $losses.insert([-scheduler \cdot diff, huber\_loss(critic, reward)])$;
  end
  $ac\_rl.apply\_gradients(losses)$;
end
$ac\_rl.save\_model()$;

Algorithm 2: Scheduling DAG tasks with AC RL selected policies.

Input: Discrete time event $t \in \mathcal{T}$
Input: AC RL selected *scheduler* configuration
Input: Slice of jobs $\mathcal{J} \in \mathcal{S}$
Input: A HPC DC $H(V^H, E^H)$
Input: Lookahead window size ($window$)
$queue\_policy, dc\_policy \leftarrow extract\_policies(scheduler)$;
$tasks \leftarrow get\_tasks\_ready\_to\_process(\mathcal{J})$;
$sort(tasks, queue\_policy)$;
for $m \in tasks$ **do**
  for $ev \leftarrow 0$ ***to*** $window - 1$ **do**
    $sort(H, dc\_policy, t + ev, window)$;
    if $do\_schedule(m, V^H, t + ev, window)$ **then**
      break;
    end
  end
end

4. Evaluation

The evaluation of our prototype is based on two simulation scenarios. While the first one use real inputs from scientific workflows, the second scenario is based on synthetic data generated to analyze the model's usage with distinct workloads. Both scenarios were simulated on a server equipped with 2 Intel(R) Xeon(R) Silver 4214 CPU @ 2.20 GHz totaling 24 cores and 48 threads, with 128 GB RAM. The model was trained on a GPU NVIDIA RTX 3090 with 24 GB of GDDR6X memory and 10 496 CUDA cores.

4.1. Simulation scenarios and parameters

Scenario I: we used the workflows profiles of real scientific applications [33,34] to generate two workload sets with 100 and 300 DAGs, representing low- and high-usage of DC resources. Each DAG has a distinct structure resulting on a heterogeneous configuration in terms of number of composing tasks, length of critical paths, and dependencies between internal layers. The jobs (100 or 300) were generated using a normal distribution for defining the arrival time of each composing task with $\mathcal{T} = 1000$. Intuitively, the scenario with 100 DAGs has lower DC usage when compared with 300 DAGs.

Scenario II: two set of DAGs were generated varying the number of edges, which represents the DAGs dependencies. Each job is composed of [1, 10] tasks following an uniform distribution, and 1000 jobs were generated using a normal distribution for defining the arrival time of each composing task with $\mathcal{T} = 1000$. We applied a configuration termed *connectivity* to represent the probability of existing a dependency between any two tasks composing the job. Specifically, two connectivity-based scenarios were prepared with 20% and 100% of probability. A connectivity of 20% indicates that any two tasks have up to 20% of chance to have a relationship (indicated by an edge). In turn, the 100% connectivity results on dense DAGs with multiple dependencies between each DAG's nodes. The procedure is executed until composing a DAG. By increasing the connectivity, we expect to highlight the importance of a DAG-aware scheduling.

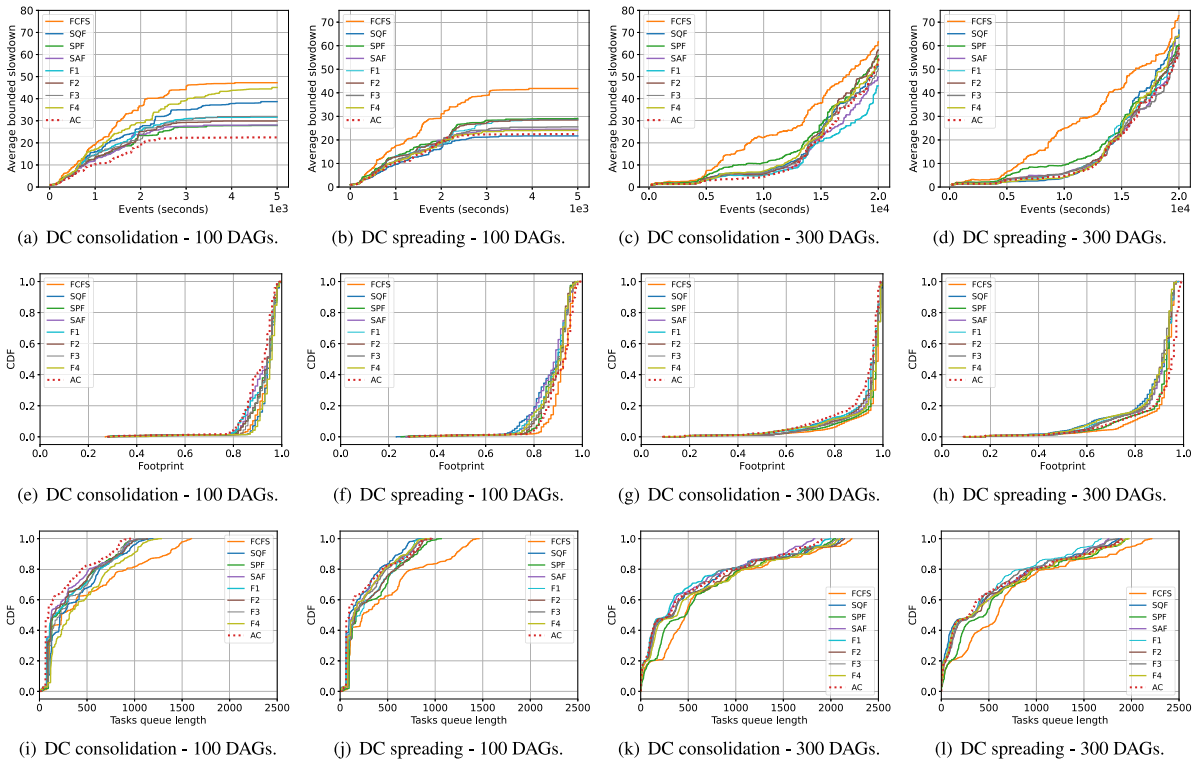
The DC is composed of 64 homogeneous servers equipped with 16 cores, and interconnected by a tree topology (with depth 3). For all scenarios, a task requests up to a full server capacity uniformly defined. Finally, we selected the 100 DAGs configuration from Scenario I for training and validating the AC RL prototype, and used the trained neural network model with all configurations to demonstrate adaptability to different workloads.

4.2. Results and discussions

The results for workflow-based simulations (Scenario I) are presented in Fig. 3, while the data for connectivity-based simulations (Scenario II) is summarized in Fig. 4. For the bounded slowdown metric, we present the average evolution, while for DC footprint and tasks queue we used Cumulative Distribution Functions (CDFs). After training, the AC RL scheduler is compared with FCFS, SAF, SPT, SQF, F1, F2, F3, and F4 queue policies. Moreover, we individually analyze the combination of such policies with DC resources consolidation and spreading policies. Finally, although the jobs arrival time follows a normal distribution up to 1000 discrete events, the graphs show longer times due to the formation of queues. It is worthwhile to highlight that the these policies compose the upper-bound limit in terms of performance, since the AC RL should learn, at most, to mimic the best performance obtained for a combination of scheduling policies for a given scenario (workloads and DC residual capacities). However, since the model can dynamically choose combinations of policies at each distinct time event, it has the potential of achieving a better performance in the long term when compared to any single combination of policies being used at all times.

4.2.1. Scenario I: scientific workflows

Fig. 3 presents the results obtained scheduling 100 and 300 scientific workflows. We split the presentation guided by the DC's objective, termed consolidation and spreading. Initially, the average bounded slowdowns are compared in Figs. 3(a)–3(d). Although imposing a lower load, the 100 DAGs scenario still presents an increasing on the average bounded slowdown for



(a) DC consolidation - 100 DAGs. (b) DC spreading - 100 DAGs. (c) DC consolidation - 300 DAGs. (d) DC spreading - 300 DAGs.

(e) DC consolidation - 100 DAGs. (f) DC spreading - 100 DAGs. (g) DC consolidation - 300 DAGs. (h) DC spreading - 300 DAGs.

(i) DC consolidation - 100 DAGs. (j) DC spreading - 100 DAGs. (k) DC consolidation - 300 DAGs. (l) DC spreading - 300 DAGs.

Fig. 3. Results for 100 and 300 workflow-based jobs submitted to a 64-servers DC.

some policies. Specifically, the FCFS policy increased the metric in both consolidation and spreading scenarios. In general, the other tasks ordering policies reduced the average slowdown when combined with a DC spreading approach. Specifically about the AC RL, it achieved the lowest values for the consolidation scenario (Fig. 3(a)) and obtained comparable results with SQF for the spreading one (Fig. 3(b)). As expected, the overall average slowdown values increased when submitting 300 DAGs (Figs. 3(c) and 3(d)). The AC RL results remain competitive with the best-case policies: F1 and SAF when combined with consolidation approach; F1, F3 and SAF for spreading case.

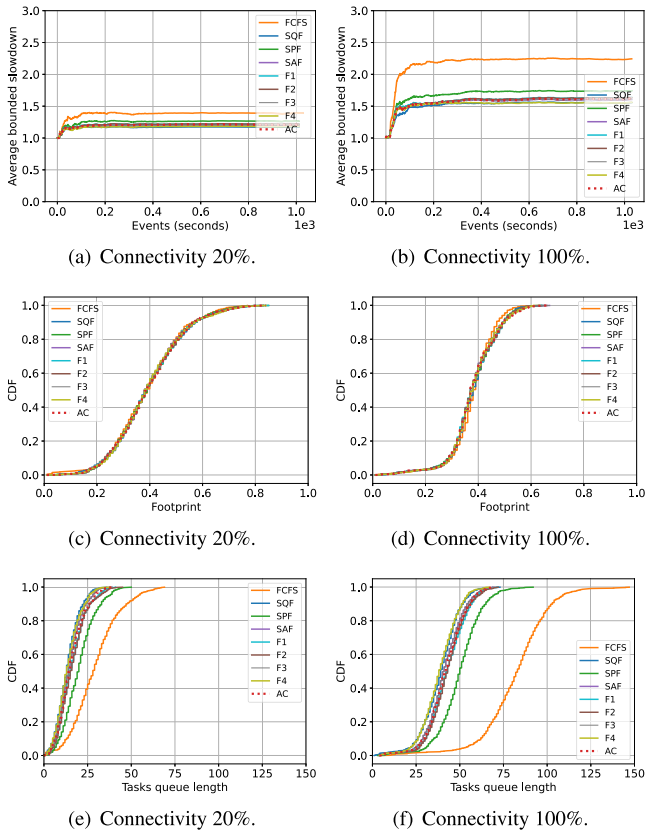
In turn, the footprint and task queue metrics represent the DC administrator's perspective. The discussion follows the same organization: we split the results into consolidation and spreading approaches. Although all policies achieved 100% of DC usage, it is important to highlight that the AC RL learned to reduce the footprint when compared to consolidation-based scenarios (Figs. 3(e) and 3(g)). However, when comparing with spreading-based scenarios (Figs. 3(f) and 3(h)) the AC RL increased the overall DC usage more rapidly when compared with other policies (an exception is perceived for FCFS). Finally, Figs. 3(i) and 3(j) summarize the queue length for 100 DAGs scenario, and Figs. 3(k) and 3(l) for the 300 DAGs one. For all scenarios, the AC RL based scheduler learned to decrease the queue length, and eventually obtained values competitive with F1, SQF, and F3.

An optimal policy should keep a lower average bounded slowdown and queue length, while efficiently using the DC resources. However, the results demonstrated that there is no *silver bullet* policy; instead the policies are dependent on the workload behavior and DC residual capacities. This fact justifies the usage of AC RL to learn which policy is adequate for scheduling a subset of tasks, as corroborated by the results obtained by our prototype. Finally, as detailed in Section 4.1 it is noteworthy that the model was trained with a separated 100 DAGs dataset, and the results indicate the model was able to generalize to a large dataset (composed of 300 DAGs). In this context, the following simulation results discuss the model's scalability and generalization with heterogeneous jobs.

4.2.2. Scenario II: scalability and heterogeneous jobs

The second simulation scenario analyzes the submission of 1000 DAGs, each one with a distinct configuration in terms of resource usage, arrival time, and critical path length (the details are presented in Section 4.1). Fig. 4 presents the AC RL results compared with all polices jointly executed with a consolidation approach. The results with spreading are omitted, since the consolidation-based values outperform their spreading counterparts. We split the presentation according to the DAGs composition: 20% or 100% of connectivity probability among tasks from the same DAG. In this sense, Figs. 4(a), 4(c), and 4(e) present the results for 20% scenario, while Figs. 4(b), 4(d), and 4(f) summarize the results for 100% connectivity.

Even increasing the connectivity degree, the queueing policies kept low values for average bounded slowdown (Figs. 4(a) and 4(b)). While FCFS obtained the worst-case results, the AC RL approach obtained competitive results with the best-case policies. In parallel, the AC RL also learned how to improve the DC usage (Figs. 4(c) and 4(d)). For the scenario with 20% of connectivity, the AC RL learned to use up to 83% of the DC for hosting the DAGs, while for the 100% of connectivity it achieved up to 68% of DC usage. This decreasing phenomena is explained by analyzing the task queue (Figs. 4(e) and 4(f)). For the 100% connectivity scenario, although the DC has enough resources for hosting more DAGs, the composing tasks are queued waiting for dependencies execution. In fact, these results highlight how challenging it is to schedule DAGs aware of dependencies, aiming at decreasing the overall average slowdown. Finally, it is evident that although the AC RL model learned to decrease the average slowdown while keeping the DC usage, it queued more tasks when compared to traditional approaches. This situation must be analyzed considering the frequency of policies selection, as discussed in the following section.



(a) Connectivity 20%. (b) Connectivity 100%.

(c) Connectivity 20%. (d) Connectivity 100%.

(e) Connectivity 20%. (f) Connectivity 100%.

Fig. 4. Results for 1000 jobs submitted to a 64-servers DC. Each job is composed of up to 10 tasks, and the DAGs are generated with an interconnection probability of 0.2 or 1.0 between layers.

4.2.3. Schedulers selection

Instead of proposing a new scheduling policy, the AC RL learns to find an efficient combination of algorithms for task queue ordering and DC resources ordering for performing the DAGs scheduling. As discussed in Section 3, the model learns the appropriate combination for each queue slice. In this sense, it is expected that the AC RL will select different combinations guided by the DAGs workloads and DC status. Figs. 5(a)–5(d) summarize the frequency of selection for each simulation scenario, where *C* denotes the consolidation policy and *S* indicates spreading.

In general, Fig. 5 demonstrates a dominance of F3 with DC consolidation followed by F1 with DC spreading. Eventually, different options were selected, even FCFS which obtained the worst performance in long term for both Scenarios I and II. The model sometimes opted for suboptimal choices as the decision is taken considering just a queue slice without full knowledge of the queue components. However, it is evident that the AC RL learned how to combine policies from distinct perspectives (which eventually represents a dichotomy in decision-making).

Another key observation extracted from Fig. 5 is the generalization capacity of the AC RL model. Without the need to perform a new model training, the AC RL was able to identify different (and better, in most cases) options for each load scenario (varying the number of tasks and connectivity). When a higher load is applied to the scenario (comparing Figs. 5(a) and 5(b)), new schedulers combinations were selected, to accommodate the dynamic DC status and queue composition. In turn, even increasing the DAG connectivity (including more dependencies), the AC RL was able to identify the appropriated combination of DC resources and tasks queue policy, and in this case, selecting essentially the same subset of policies (Figs. 5(c) and 5(d)).

The simulations results indicated that such heterogeneous scenario (i.e., distinct DAGs compositions, resources and workloads) demands careful choices of alternatives to select the appropriate scheduling policy and where there is no *silver bullet* scheduler that is best for all cases. The results highlighted that the AC RL, in long term, can learn which combination of policies must be used to maximize the proposed reward function. Our prototype, following the specialized literature, aims at decreasing the overall bounded slowdown (bsld) while increasing the number of accepted tasks. This objective guided the actor's decisions (as demonstrated by Fig. 5) which eventually selected consolidation and spreading DC policies, as well as distinct tasks policies. Finally, although the use of a different reward function may lead to new results when comparing the performance of the AC RL model, the overall prototype and training process remains the same, as well as the upper-bound limits of performance, given by the preselected policies.

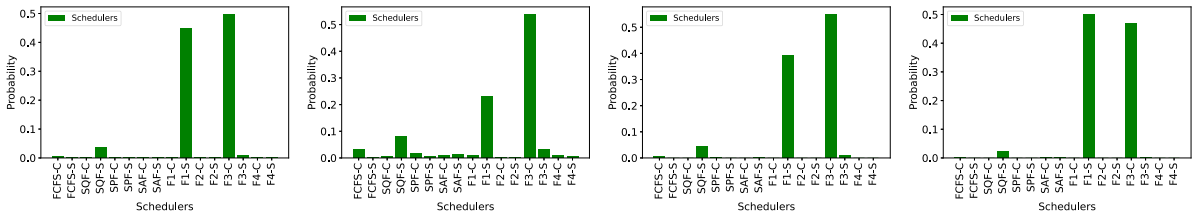
5. Related work

The scheduling of tasks atop HPC infrastructures is a well-known problem in specialized literature. Specifically, ML techniques are commonly used to improve the performance of existing policies and to propose new alternatives. In this sense, we review some ML-based works, focusing on the use RL, mainly AC, which seems to be still evolving nowadays.

Several studies used RL and DRL to build task schedulers. DeepRM is an attempt to build agents that learn to schedule in order to minimize the slowdown. The proposal uses matrices to represent a Gantt chart and queued jobs, performing the RL on such data, and as a result, proposing a new policy. The authors of Spear [35] combined DRL and Monte Carlo tree search to schedule complex DAGs. After training, Spear outperformed the traditional scheduler Graphene [12] regarding makespan reduction. Spear clearly demonstrated the potential of applying DRL to improve user-designed policies, using DRL in expansion and simulation phases of Monte Carlo tree search. In [24], the authors aim to use a method for parallelizing task scheduling via DRL in a scalable manner. They try to solve the high dimensionality issue of traditional parallel computing environments by hooking the DRL method to another method which is a multi-task decision, thereby matching the output branches of the multi-task learning to parallel scheduling tasks.

The work [29] introduced regression methods to propose new policies (termed F1, F2, F3, and F4, used for composing the evaluation process - Section 4). In turn, the work [36] presented a DRL-based task scheduling algorithm. The agent receives as input the current state of the environment (i.e., the remaining resources, previous allocations) and information about the task to be scheduled, and after processing it schedules the task for some resource or move to the next unit of time. The work presents results close to or even better than state-of-the-art heuristics in the slowdown and completion time metrics. Following this line, the works [37,38] presented a similar approach, in which the scheduler decides between the request allocation or the passage to the next unit of time considering the improvement of energy-related metrics and decision-making.

The work [39] proposed a network-aware algorithm for DC resource allocation. It utilizes a framework based on RL techniques and graph neural networks to assimilate network-aware allocation policies. In turn, an approach for tackling the traditional job-shop problem using DRL was proposed in [40]. The authors design a state representation which has a reward function resembling a traditional sparse makespan minimization objective from methods to combinatorial optimization problems. The DRL-based



(a) 100 DAGs - Scenario I. (b) 300 DAGs - Scenario I. (c) Connectivity 20% - Scenario II. (d) Connectivity 100% - Scenario II.

Fig. 5. Frequency of policies selection for all scenarios.

approach outperformed the traditional fixed priority policies. Finally, Legrand et al. [41] motivated the composition of mixed polices based on various features extracted from the jobs.

Our proposal advances the field by proposing the use of AC RL to scheduler DAGs, and not only individual stand-alone tasks. Moreover, the AC RL reduces the administrator interference on configurations (which is represented by the reward function - Section 3.3), as well as combining both jobs and DC perspectives for composing the environment (as discussed in Section 2.2). We selected a subset of existing state-of-the-art queue policies for composition of the evaluation prototype, including ML-based policies. However, it is worthwhile to mention that any aforementioned proposal can be potentially used for composing the candidates pool of our AC RL model.

6. Conclusion

The efficient use of High-Performance Computing (HPC) infrastructures, independently of scale, are directly dependent from scheduling decisions taken by the RJMS. The realization of an optimized schedule is a problem that fits in the NP-Hard class. The scheduling problem is not highly complex due only to its combinatorial nature, but also due to the high number of contexts where it is inserted (e.g., tasks, workload, dependencies) which may have different definitions of the optimal solution. A relevant feature that further increases the complexity of the scheduling problem is the communication between tasks, composing a Directed Acyclic Graph (DAG), which requires a new range of resources to be taken into account in scheduling and, even more, inserting a concept of strong dependency between tasks, increasing the impact of scaling on system performance. In this work, the intersection between DAGs scheduling and machine learning is presented, exploring mainly the niche of communicating tasks combined with Actor–Critic (AC) Reinforcement Learning (RL). However, instead of proposing a new *silver bullet* queueing policy, we claim that the existing approaches from specialized literature are efficient enough, however the use of each one is highly dependent on the HPC infrastructure workload and DAGs structure. In this sense, we proposed the use of AC RL to learn the appropriate combination between task and DC resources ordering policies. An experimental prototype was implemented and analyzed to identify the benefits and drawbacks. In summary, the AC RL dynamically adapted the policies selection for different workload contexts, eventually outperforming the counterparts schedulers.

This promising line opens new research opportunities for future work. First and foremost, we aim to integrate the network topology details and load into AC RL model for improving the decision-making with network status information (e.g., congested paths, packet queues). Secondly, the AC RL efficiency is dependent mainly from the reward function. We empirically proposed a reward function based on average bounded slowdown and acceptance ratio. As future work, we will look into other functions that jointly represent the users and HPC DC administrators (e.g., energy-related aspects, data locality). Finally, different workloads and arrival distribution functions can lead to challenging scenarios, opening new opportunities to improve the AC RL model.

CRediT authorship contribution statement

Guilherme Piêgas Koslovski: Writing – original draft, Designed and Developed the AC RL prototype, Executed the experimental scenario,Analyzed the experimental data. **Kleiton Pereira:** Writing – original draft, Designed the AC RL prototype, Analyzed the experimental data. **Paulo Roberto Albuquerque:** Writing – original draft, Designed the AC RL prototype, Analyzed the experimental data.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request

Acknowledgments

This work was funded by the National Council for Scientific and Technological Development (CNPq), Brazil, the Santa Catarina State Research and Innovation Support Foundation (FAPESC), Brazil, Santa Catarina State University (UDESC), Brazil, and developed at Laboratory of Parallel and Distributed Processing (LabP2D). All authors approved the version of the manuscript to be published.

References

[1] D.E. Bernholdt, S. Boehm, G. Bosilca, M.G. Venkata, R.E. Grant, T. Naughton, H.P. Pritchard, M. Schulz, G.R. Vallee, A survey of MPI usage in the US exascale computing project, Concurr. Comput.: Pract. Exper. 32 (3) (2020) e4851, http://dx.doi.org/10.1002/cpe.4851, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.4851.

[2] D. Kothe, S. Lee, I. Qualters, Exascale computing in the United States, Comput. Sci. Eng. 21 (1) (2019) 17–29, http://dx.doi.org/10.1109/MCSE.2018.2875366.

[3] P. Brucker, Scheduling algorithms, J. Oper. Res. Soc. 50 (1999) 774.

[4] J.K. Lenstra, A.R. Kan, Computational complexity of discrete optimization problems, in: Annals of Discrete Mathematics, Vol. 4, Elsevier, 1979, pp. 121–140.

[5] T. Gonzalez, S. Sahni, Flowshop and jobshop schedules: complexity and approximation, Oper. Res. 26 (1) (1978) 36–52.

[6] E. Amaldi, S. Coniglio, A.M. Koster, M. Tieves, On the computational complexity of the virtual network embedding problem, Electron. Notes Discrete Math. 52 (2016) 213–220, http://dx.doi.org/10.1016/j.endm.2016.03.028.

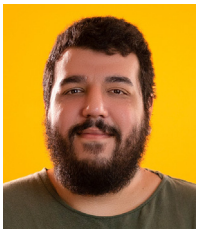
[7] M.R. Garey, D.S. Johnson, R. Sethi, The complexity of flowshop and jobshop scheduling, Math. Oper. Res. 1 (2) (1976) 117–129.

[8] M. Noormohammadpour, C.S. Raghavendra, Datacenter traffic control: Understanding techniques and tradeoffs, IEEE Commun. Surv. Tutor. 20 (2) (2018) 1492–1525, http://dx.doi.org/10.1109/COMST.2017.2782753.

[9] J. Dongarra, P. Luszczek, in: D. Padua (Ed.), TOP500, Springer US, Boston, MA, 2011, pp. 2055–2057, http://dx.doi.org/10.1007/978-0-387-09766-4_157.

[10] A.W. Mu'alem, D.G. Feitelson, Utilization, predictability, workloads, and user runtime estimates in scheduling the IBM SP2 with backfilling, IEEE Trans. Parallel Distrib. Syst. 12 (6) (2001) 529–543.
[11] D. Carastan-Santos, R.Y. De Camargo, D. Trystram, S. Zrigui, One can only gain by replacing EASY backfilling: A simple scheduling policies case study, in: 2019 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID), IEEE, 2019, pp. 1–10.
[12] R. Grandl, S. Kandula, S. Rao, A. Akella, J. Kulkarni, {GrapheNE}: Packing and {dependency-aware} scheduling for {data-parallel} clusters, in: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), 2016, pp. 81–97.
[13] L.R. Rodrigues, G.P. Koslovski, M. Pasin, M.A. Pillon, O.C. Alves, C.C. Miers, Time-constrained and network-aware containers scheduling in GPU era, Future Gener. Comput. Syst. 117 (2021) 72–86, http://dx.doi.org/10.1016/j.future.2020.11.014, URL https://www.sciencedirect.com/science/article/pii/S0167739X20330387.
[14] L.L. Nesi, M.A. Pillon, M.D. de Assunção, C.C. Miers, G.P. Koslovski, Tackling virtual infrastructure allocation in cloud data centers: a GPU-accelerated framework, in: 2018 14th International Conference on Network and Service Management (CNSM), 2018, pp. 191–197.
[15] M. Demirci, A survey of machine learning applications for energy-efficient resource management in cloud computing environments, in: Machine Learning and Applications (ICMLA), 2015 IEEE 14th International Conference on, IEEE, 2015, pp. 1185–1190.
[16] I.A.T. Hashem, I. Yaqoob, N.B. Anuar, S. Mokhtar, A. Gani, S.U. Khan, The rise of "big data" on cloud computing: Review and open research issues, Inf. Syst. 47 (2015) 98–115.
[17] F. Li, B. Hu, Deepjs: Job scheduling based on deep reinforcement learning in cloud data center, in: Proceedings of the 2019 4th International Conference on Big Data and Computing, in: ICBDC 2019, ACM, New York, NY, USA, 2019, pp. 48–53, http://dx.doi.org/10.1145/3335484.3335513, URL http://doi.acm.org/10.1145/3335484.3335513.
[18] H. Yao, X. Chen, M. Li, P. Zhang, L. Wang, A novel reinforcement learning algorithm for virtual network embedding, Neurocomputing 284 (2018) 1–9, http://dx.doi.org/10.1016/j.neucom.2018.01.025, URL http://www.sciencedirect.com/science/article/pii/S0925231218300420.
[19] A. Blenk, P. Kalmbach, W. Kellerer, S. Schmid, O'zapft is: Tap your network algorithm's big data!, in: Proceedings of the Workshop on Big Data Analytics and Machine Learning for Data Communication Networks, in: Big-DAMA '17, ACM, New York, NY, USA, 2017, pp. 19–24, http://dx.doi.org/10.1145/3098593.3098597, URL http://doi.acm.org/10.1145/3098593.3098597.
[20] J. Li, X. Zhang, J. Wei, Z. Ji, Z. Wei, Garlsched: Generative adversarial deep reinforcement learning task scheduling optimization for large-scale high performance computing systems, Future Gener. Comput. Syst. 135 (2022) 259–269, http://dx.doi.org/10.1016/j.future.2022.04.032, URL https://www.sciencedirect.com/science/article/pii/S0167739X22001613.
[21] R. Boutaba, M.A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, O.M. Caicedo, A comprehensive survey on machine learning for networking: evolution, applications and research opportunities, J. Internet Serv. Appl. 9 (1) (2018) 1–99.
[22] C.-L. Liu, C.-C. Chang, C.-J. Tseng, Actor-critic deep reinforcement learning for solving job shop scheduling problems, IEEE Access 8 (2020) 71752–71762, http://dx.doi.org/10.1109/ACCESS.2020.2987820.
[23] C. Shyalika, T. Silva, A. Karunananda, Reinforcement learning in dynamic task scheduling: A review, SN Comput. Sci. 1 (6) (2020) 1–17.
[24] L. Zhang, Q. Qi, J. Wang, H. Sun, J. Liao, Multi-task deep reinforcement learning for scalable parallel task scheduling, in: 2019 IEEE International Conference on Big Data (Big Data), IEEE, 2019, pp. 2992–3001.
[25] J. Song, G.d. Veciana, S. Shakkottai, Meta-scheduling for the wireless downlink through learning with bandit feedback, in: 2020 18th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOPT), 2020, pp. 1–7.
[26] T. Jaakkola, M.I. Jordan, S.P. Singh, On the convergence of stochastic iterative dynamic programming algorithms, Neural Comput. 6 (6) (1994) 1185–1201.
[27] S. Bhatnagar, R.S. Sutton, M. Ghavamzadeh, M. Lee, Natural actor–critic algorithms, Automatica 45 (2009) 2471–2482.
[28] D.G. Feitelson, L. Rudolph, Metrics and benchmarking for parallel job scheduling, in: Workshop on Job Scheduling Strategies for Parallel Processing, Springer, 1998, pp. 1–24.
[29] D. Carastan-Santos, R.Y. de Camargo, Obtaining dynamic scheduling policies with simulation and machine learning, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '17, Association for Computing Machinery, New York, NY, USA, 2017, http://dx.doi.org/10.1145/3126908.3126955.
[30] L.L. Mentz, W.J. Loch, G.P. Koslovski, Comparative experimental analysis of docker container networking drivers, in: 2020 IEEE 9th International Conference on Cloud Networking (CloudNet), 2020, pp. 1–7, http://dx.doi.org/10.1109/CloudNet51028.2020.9335811.
[31] I. Grondman, L. Busoniu, G.A.D. Lopes, R. Babuska, A survey of actor-critic reinforcement learning: Standard and natural policy gradients, IEEE Trans. Syst. Man Cybern. C 42 (6) (2012) 1291–1307, http://dx.doi.org/10.1109/TSMCC.2012.2218595.
[32] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2017, arXiv:1412.6980.
[33] S. Bharathi, A. Chervenak, E. Deelman, G. Mehta, M.-H. Su, K. Vahi, Characterization of scientific workflows, in: 2008 Third Workshop on Workflows in Support of Large-Scale Science, 2008, pp. 1–10, http://dx.doi.org/10.1109/WORKS.2008.4723958.
[34] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, K. Vahi, Characterizing and profiling scientific workflows, Future Gener. Comput. Syst. 29 (3) (2013) 682–692, http://dx.doi.org/10.1016/j.future.2012.08.015, URL https://www.sciencedirect.com/science/article/pii/S0167739X12001732. Special Section: Recent Developments in High Performance Computing and Security.
[35] Z. Hu, J. Tu, B. Li, Spear: Optimized dependency-aware task scheduling with deep reinforcement learning, in: 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS), IEEE, 2019, pp. 2037–2046.
[36] H. Mao, M. Alizadeh, I. Menache, S. Kandula, Resource management with deep reinforcement learning, in: Proceedings of the 15th ACM Workshop on Hot Topics in Networks, HotNets '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 50–56, http://dx.doi.org/10.1145/3005745.3005750.
[37] L.C. Casagrande, G.P. Koslovski, C.C. Miers, M.A. Pillon, DeepScheduling: Grid computing job scheduler based on deep reinforcement learning, in: L. Barolli, F. Amato, F. Moscato, T. Enokido, M. Takizawa (Eds.), Advanced Information Networking and Applications, Springer International Publishing, Cham, 2020, pp. 1032–1044.
[38] L.C. Casagrande, G.P. Koslovski, C.C. Miers, M.A. Pillon, N.M. Gonzalez, Don't hurry be green: scheduling servers shutdown in grid computing with deep reinforcement learning, Int. J. Grid Util. Comput. 13 (6) (2022) 589–606, http://dx.doi.org/10.1504/IJGUC.2022.128303, URL https://www.inderscienceonline.com/doi/abs/10.1504/IJGUC.2022.128303. arXiv:https://www.inderscienceonline.com/doi/pdf/10.1504/IJGUC.2022.128303.
[39] Z. Shabka, G. Zervas, Resource allocation in disaggregated data centre systems with reinforcement learning, 2021, arXiv:2106.02412.
[40] P. Tassel, M. Gebser, K. Schekotihin, A reinforcement learning environment for job-shop scheduling, 2021, arXiv:2104.03760.
[41] A. Legrand, D. Trystram, S. Zrigui, Adapting batch scheduling to workload characteristics: What can we expect from online learning? in: 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS), 2019, pp. 686–695, http://dx.doi.org/10.1109/IPDPS.2019.00077.

Guilherme Piêgas Koslovski Professor of Computer Networks and Parallel Programming at Santa Catarina State University (UDESC) in Joinville/SC - Brazil. He received his doctorate from École Normale Supérieure at Lyon, France, the master's degree from Federal University of Santa Maria (UFSM), Brazil, and his bachelor's degree from the UFSM, Brazil, in Computer Science. Currently, his researches are related to high performance computing, virtual infrastructures, scheduling, software defined networks, and virtualization of computational and communication resources. He is the coordinator of LabP2D (Laboratory of Parallel and Distributed Processing) at UDESC.



Kleiton Pereira holds a Bachelor's degree in Computer Science from UDESC Joinville/SC - Brazil, and is currently pursuing a Master's degree in Applied Computing at the same institution. He is specialized and has done research in the fields of High-Performance Computing, Theoretical Computer Science, and Artificial Intelligence. Currently he works as a Tech Lead at a Data Science company.

Paulo Roberto Albuquerque holds a Bachelor's degree in Computer Science from Universidade do Estadode Santa Catarina at Joinville/SC - Brazil. He is currently pursuing a Master's degree in Applied Computing, in the scope of Computer Systems in the same University. He has done research in Distributed Systems, Scheduling Algorithms, Deep Machine Learning and High-Performance Computing. Currently he works as a Systems Developer & Engineering at WEG Electric Motors.